Randomized Reduction

Design and Analysis of Algorithms

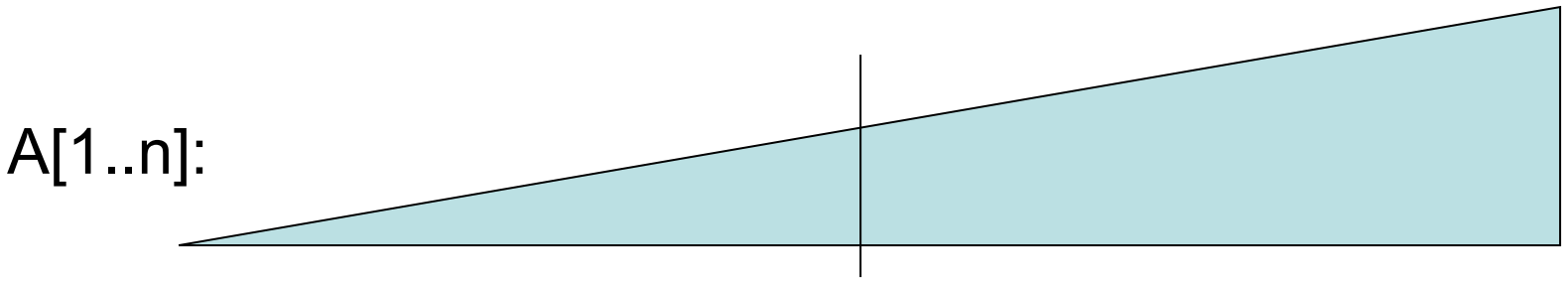
Brian C. Dean

School of Computing

Clemson University

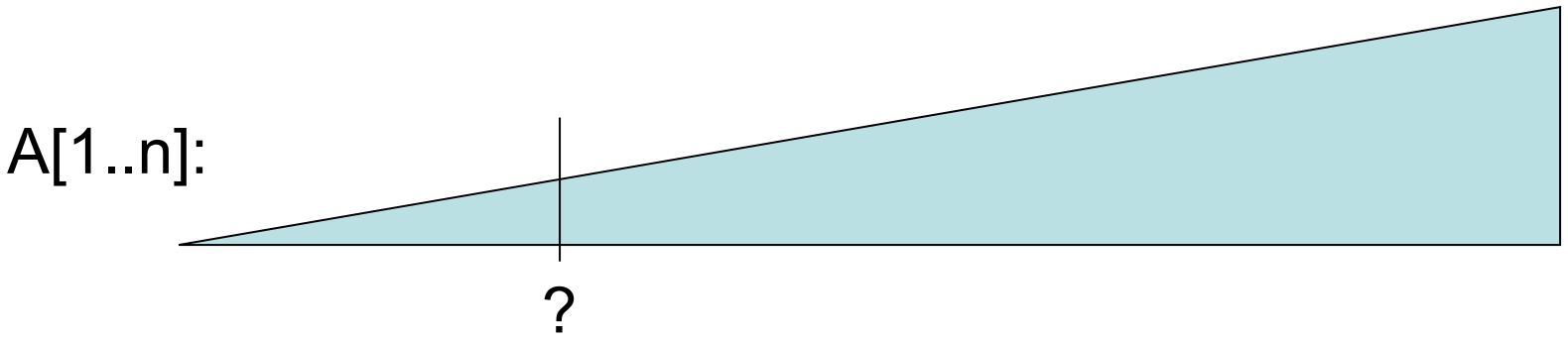
An Easy Starting Point

- Suppose we have an algorithm for which:
  - We start with a problem of size n.
  - In every iteration, effective problem size is reduced to a constant fraction of its current value.
- Then, our algorithm performs only O(log n) iterations.
- Prototypical application: binary search.



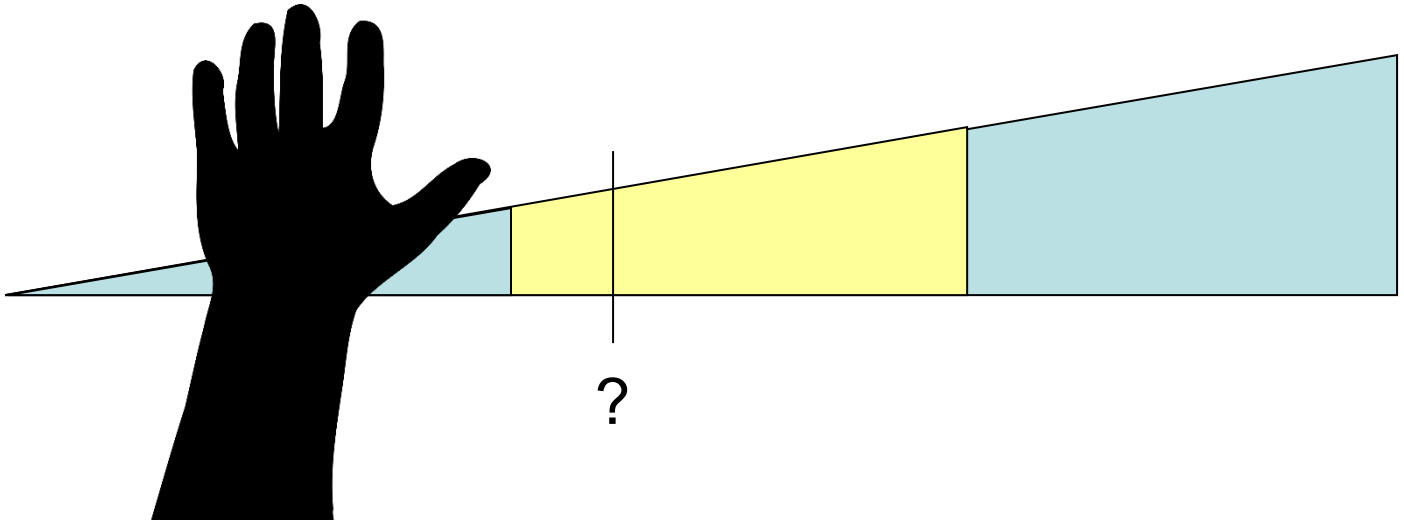
The Randomized Reduction Lemma

- Suppose we have an algorithm for which:
  - We start with a problem of size n.
  - In every iteration, effective problem size is reduced to a constant fraction of its current value **with some constant probability**.
- Then, our algorithm performs only O(log n) iterations **in expectation**.
- Prototypical application: randomized binary search.



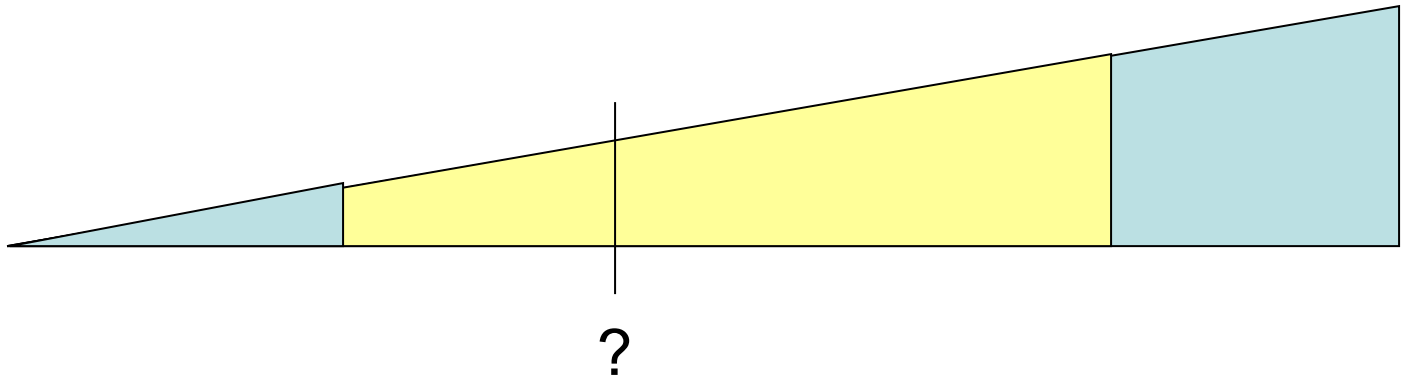
Example: Randomized Binary Search

- If we choose to compare against an element in the middle third of our array (this happens with probability 1/3), then our problem will be reduced to ≤ 2/3 of its original size.
- Thus, randomized binary search takes O(log n) expected time.



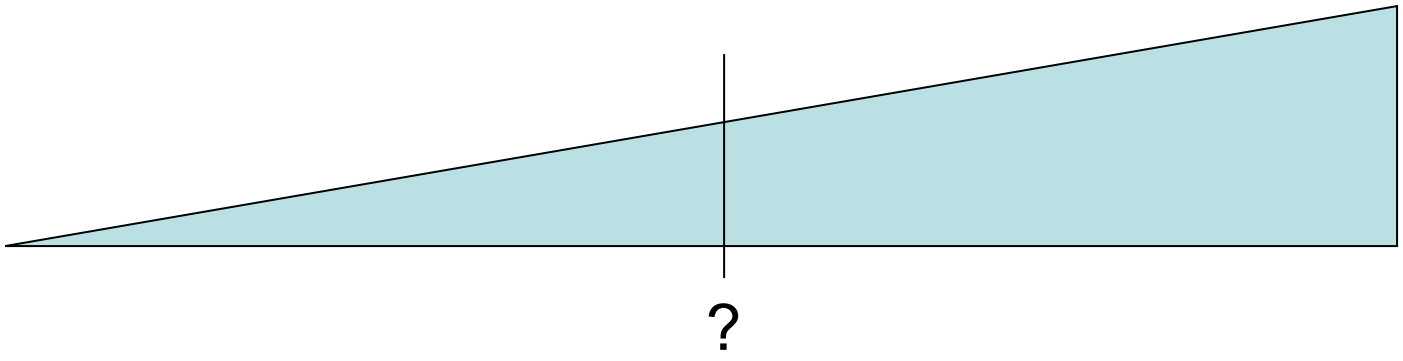
Example: Randomized Binary Search

- If we choose to compare against an element in the middle half of our array (this happens with probability 1/2), then our problem will be reduced to ≤ 3/4 of its original size.
- Thus, randomized binary search takes O(log n) expected time.



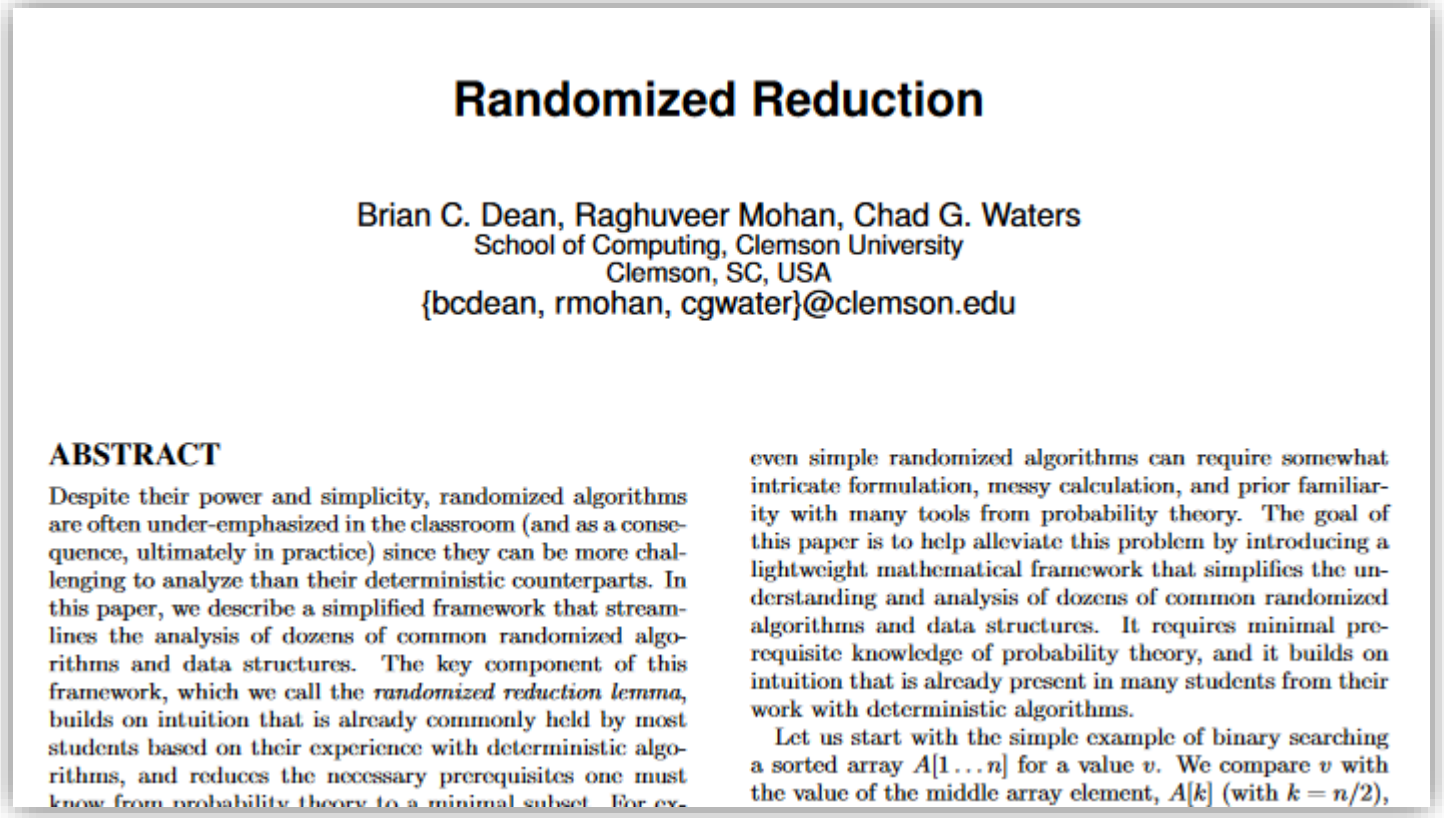
Why Not Reducing by ½?

- If we choose to compare against the exact middle element (this happens with probability only 1/n ☹), then our problem will be reduced to ≤ 1/2 of its original size.
- This doesn't satisfy the conditions of randomized reduction…



“Randomized Reduction”

- The “randomized reduction” lemma name is from a paper I published advocating the use of this approach as a simple tool for analyzing a wide range of randomized algorithms and data structures.



Randomized Reduction

Brian C. Dean, Raghuveer Mohan, Chad G. Waters
School of Computing, Clemson University
Clemson, SC, USA
{bcdean, rmohan, cgwater}@clemson.edu

ABSTRACT

Despite their power and simplicity, randomized algorithms are often under-emphasized in the classroom (and as a consequence, ultimately in practice) since they can be more challenging to analyze than their deterministic counterparts. In this paper, we describe a simplified framework that streamlines the analysis of dozens of common randomized algorithms and data structures. The key component of this framework, which we call the *randomized reduction lemma*, builds on intuition that is already commonly held by most students based on their experience with deterministic algorithms, and reduces the necessary prerequisites one must know from probability theory to a minimal subset. For ex-

even simple randomized algorithms can require somewhat intricate formulation, messy calculation, and prior familiarity with many tools from probability theory. The goal of this paper is to help alleviate this problem by introducing a lightweight mathematical framework that simplifies the understanding and analysis of dozens of common randomized algorithms and data structures. It requires minimal prerequisite knowledge of probability theory, and it builds on intuition that is already present in many students from their work with deterministic algorithms.

Let us start with the simple example of binary searching a sorted array $A[1 \ldots n]$ for a value v. We compare v with the value of the middle array element, $A[k]$ (with $k = n/2$),